



Поиск моделей, наборов данных, пользователей...



вэньчжоуу / **YoungAi-DeepSeek-V4.1-Flash**



Нравится 7



Генерация текста



GGUF



Английский



Китайский

глубокий взгляд

Команда экспертов

векторное квантование

cuda

dgx-искра

встроенное устройство



Лицензия: mit



→ Скопировать в корзину **НОВУЮ**



Карточка модели

Файлы

xet



Сообщество 1

Загрузки за последний месяц

-

Для этой модели загрузки не отслеживаются. [Как отслеживать](#)

⚡ Поставщики логического вывода **НОВОЕ**



Генерация текста

Эта модель не развернута ни одним провайдером логического вывода.



1

Обратитесь за поддержкой к провайдеру

🌳 Дерево моделей для wenzhouwu/YoungAi-DeepSeek-V4.1-Flash

Базовая модель

[deepseek-ai/DeepSeek-V4.1-Flash](#)



Квантованный (98)

[эта модель](#)



🔗 YoungAi — DeepSeek V4.1 Flash на одном DGX Spark

Английский · [Китайский](#)

“Одна коробка на 128 ГБ. Три файла. Модель, официальный контрольный файл которой весит 510 ГБ.

① *общая база на 113,6 ГБ* → ② *доменный сайдкар ~40 МБ* → ③ *файл после обучения, который пересчитывается каждую ночь.*”

ds4 — это собственный механизм логического вывода на C/CUDA, а также автономный набор инструментов, который запускает DeepSeek V4.1 Flash на одном графическом процессоре NVIDIA DGX Spark (GB10, 128 ГБ объединенной памяти). Все, что здесь описано, — наша собственная разработка: формат векторного квантования, решатели, которые создают sidecar и файл после обучения, ядра CUDA и метрики, по которым мы все оцениваем. Архитектура модели соответствует DeepSeek и здесь не описывается — читайте официальный релиз.

Как читать эту страницу. Она состоит из пяти этапов; остановитесь, когда найдете то, что вам нужно.

- 1. Цифры — что работает, с какой скоростью, насколько близко к оригиналу.
- 2. Три убеждения — почему система устроена именно так.
- 3. Архитектура — три файла и их взаимосвязь.
- 4. Алгоритмы — каждый элемент проходит путь *одно предложение* → *интуиция* → *математика* → *инженерия* → *доказательства*.
- 5. Результаты, что не сработало, как это запустить, ограничения.

🔗 1. Цифры

Один DGX Spark, ① базовая версия + ② дополнительная версия измеряемого домена.

Постоянно присутствующий в памяти	Модель 113,6 ГБ + ~40 МБ sidecar (≈ 1,6 бита на вес всего файла)
Оставлен на SSD, не тронут	203 ГБ n-граммных таблиц памяти модели, оригинальный FP8, ~6 КБ на токен

Минимальная ставка восстановления $\Sigma \min$ — финансы / код / право / медицина / наука	0.745 / 0.803 / 0.760 / 0.739 / 0.753
Тот же топ-1, что и в оригинале — финансы / программирование / юриспруденция / медицина / наука	74.6% / 82.3% / 76.4% / 72.8% / 74.9%
Общий английский (WikiText-2) с любым из пяти дополнительных модулей	$\Sigma \min$ 0,787–0,801, Same top-1 80,7–82,8% (без учета базы 0,769 / 78,5%)
Prefill, подсказка на 12,5 тыс. токенов	1055 токенов/с · 940 токенов/с при реальном запросе агента на 14,1 тыс. токенов · 671 токен/с при запросе на 106,7 тыс. токенов через сервер
Расшифровать, обычная жадность	30,7 токенов/с при коротком контексте · 30 токенов/с при контексте 14k
Decode, спекулятивный, жадный (включен по умолчанию)	45,5 токенов/с при реальном запросе агента на 14 000 токенов · 42,4 токенов/с при запросе, который никогда не использовался для настройки
Decode, спекулятивная модель, сэмплированная на уровне temperature 1.0 (по умолчанию в чате)	34–37 токенов/с при том же запросе к агенту (четыре запуска) · 33–42 токенов/с при запросе без настройки (три запуска); декодирование с простой выборкой — 30. Каждый запуск с выборкой — это отдельный текст, поэтому процент принятия и скорость варьируются от запуска к запуску; при выборке с отбраковкой распределение остается неизменным.
Контекст	1 млн токенов, взятых из собственных метаданных модели; полный кэш KV размером 1 млн токенов занимает 0,89 ГБ

Коэффициент восстановления = $\Sigma_v \min(p_{\text{original}}(v), p_{\text{ours}}(v))$, усреднённое по позициям: доля массы вероятности следующего токена, в отношении которой

наша модель и исходная модель совпадают. 1,0 означает идентичные распределения. Все показатели определены в [§4.5](#).

🔗 2. Три убеждения

Каждое убеждение, приведенное ниже, сначала сформулировано, затем подкреплено результатами наших измерений и связано с тем, что мы создали.

🔗 2.1 В каждом домене есть сильно коррелирующая закономерность — найдите ее, и несколько мегабайт восстановят многое

Претензия. С точки зрения одной области, повреждение квантования не является однородным шумом. Текст из одного домена использует узкий набор направлений, и ошибка в этих направлениях может быть обнаружена на основе нескольких тысяч токенов. Исправление этого восстанавливает больше возможностей на байт, чем когда-либо могло быть добавление битов к весам.

То, что мы измерили.

- **Пять областей, один рецепт.** Финансы, программирование, юриспруденция, медицина и наука — каждая из этих областей получает дополнительный модуль от одного и того же решателя. На собственном выделенном тексте каждый дополнительный модуль улучшает результат на 1-е место на **+2,8–3,7 pp** и сокращает среднее KL на **16–28%** (таблицы в [§5.1](#)).
- **Не вредит общему тексту.** В английской Википедии каждый из пяти сайдкаров повышает показатель Same top-1 с 78,5 % до **80,7–82,8 %** и $\Sigma \min$ с 0,769 до **0,787–0,801**.
- **Leverage.** +3 pp — это примерно 0,15 дополнительных битов на вес, что дает около 11 ГБ дополнительного экспертного хранилища. Сайдкар, который это обеспечивает, весит около **40 МБ**.

Что мы создали. Базу, которая никогда не взаимодействует с доменными данными, плюс небольшой дополнительный модуль для каждого домена ([§4.2](#)).

🔗 2.2 Законы масштабирования реальны, а знания распределяются как хэш

Претензия. Возможность следует за сохраняемыми вами байтами. Вы не можете обмануть закон масштабирования, решив, что некоторые эксперты, слои или матрицы "имеют большее значение" — веса не показывают такой структуры, а маршрутизация ведет себя как хэш токена. Отдача от оборудования тоже падает: на вторую коробку покупается меньше, чем она стоит.

То, что мы измерили.

- **У гирь нет фаворитов.** На реальной контрольной точке 384 эксперта слоя имеют одинаковую относительную ошибку квантования с точностью до **1,9%**; энергия эксперта отличается не более чем на **1,25 ×**; уровни отличаются на **3%** по Σw^2 ; три матрицы эксперта разделены **33.5 / 33.3 / 33.2**. Перемещение одного бита из группы В в группу А окупается, только если значение А по крайней мере **в 1,189 раз** больше, потому что каждый дополнительный бит на 8 весов умножает ошибку на $1/1.189$ (измерено 0.201 / 0.171 / 0.143 при 11 / 12 / 13 битах). Ничто не пересекает эту линию, поэтому оптимальное «динамическое» распределение является просто равномерным. Распределение по слоям и экспертам не дает никаких преимуществ.
- **Маршрутизация похожа на хеширование.** После квантования доля токенов, для которых выбраны 6 экспертов, в точности совпадающих с первоначальным выбором, снижается до 1,6–28 % в средних и глубоких слоях, а систематическая ошибка на уровне экспертов объясняет только 3–22 % отклонений — остальное приходится на каждый токен в отдельности. Последовательные токены почти не пересекаются с экспертами: 1, 2, 3, 4, 5 токенов пересекаются с 6, 10, 0, 13, 3, 17, 4, 21, 7 различными экспертами.
- **Обрезка — это скрытое решение, принимаемое на уровне предметной области.** Единственное отклонение в 20–80 раз, которое мы обнаружили, связано с тем, как часто эксперты перенаправляются *в рамках заданного корпуса*: измените корпус, и отклонение изменится. Чтобы уместить модель в 100 ГБ, пришлось бы оставить 130 из 384 экспертов на каждом уровне.

- **Чем больше битов, тем меньше отдача** (только эксперты, линейка из английской Википедии): 1,0 / 1,5 / 2,0 / 2,5 / 3,0 бита на вес $\rightarrow \Sigma_{\min} 0,445 / 0,746 / 0,848 / 0,900 / 0,927$. Последний полубайт дает +0,027.
- **Чем больше блоков, тем меньше отдача.** Декодирование ограничено количеством байтов, считываемых за один токен. Разделение слоев между машинами добавляет сетевой переход для каждого токена (исходный код ds4 показал снижение производительности на 19 % на двух компьютерах Mac); тензорный параллелизм удваивает количество байтов, но в нашем тесте на двух машинах прирост составил всего 1,25 раза, потому что all-reduce съедает половину прироста. Для сравнения: в конфигурации из двух Spark в сообществе используется более компактная V4 Flash (284B) в режиме FP8 с TP=2 и спекулятивным декодированием со скоростью ~41 токен/с; один Spark здесь работает с V4.1 со скоростью 42–46.

Что мы создали. Один блок. Каждый байт расходуется равномерно. Все остальное зависит от сайдкара и пост-обучения, а не от топологии.

🔗 2.3 Для реальной работы требуется пост-обучение — и оно должно выполняться на вашем компьютере

Претензия. В реальной бизнес-работе - в реальной кодовой базе, в реальном торговом центре — открытые модели недостаточно хороши "из коробки", и точность квантования не может этого исправить: идеальная копия оригинала наследует ошибки оригинала. Модель должна извлекать уроки из своего собственного развертывания, и эти данные (позиции, обзоры, исходный код) никогда не должны покидать машину.

Что мы измерили. Тестовый стенд: конвейер торговых агентов, обслуживаемый ds4 (прогноз рынка, объединение новостей, выбор акций, отчеты о решениях финансового директора; запросы на 9–132 тыс. токенов). **12 из 12** целевых цен, установленных моделью, были выше фактического максимума следующего дня. По одному запросу на прогноз рынка модель выдала ответ «рынок растет»; в тот день 1891 акция выросла, а 3563 — упали.

Что мы создали. Третий файл, созданный на основе запросов текущего дня и добавленный поверх остальных, не затрагивает ① и ② (\$4.3). Механизм работает,

но пока не обобщает данные по разным дням — см. §8.

🔗 3. Архитектура: базовая модель + sidecar + файл для постобработки

	what it is	how often it changes
① base GGUF	113.6 GB, general, zero corpus	quantized once for all
② sidecar	~40 MB per domain (<code>--zchain</code>)	solved once per domain
③ post-train	same format as ② (<code>--posttrain</code>)	re-solved nightly

each routed-expert weight row: $g_{\text{eff}} = g_{\text{base}} \times s_{\text{sidecar}} \times s_{\text{posttrain}}$
each router: selection score $+= \Delta b_{\text{sidecar}}$

- **① сама по себе является полноценной моделью.** ② и ③ лишь изменяют масштаб уже существующих строк и корректируют маршрутизатор. Они не добавляют новых слоев; ядра с ними и без них работают одинаково.
- **Практически нулевая стоимость выполнения.** ② добавляет ~0,6 МБ чтения на каждый декодированный токен, что составляет около 0,01 % от того, что считывает базовая модель.
- **Пары идентифицируются по отпечаткам пальцев.** ③ хранит хэш ②, с которым он был сопоставлен (`base.fnv`); механизм отклоняет несовпадающие пары. Без этой проверки устаревший ③ будет работать нормально и выдавать неверные числа.

```
./bin/ds4-server --cuda -m base.gguf --zchain sidecar_dir/ [--posttrain ...]
```

🔗 4. Алгоритмы

🔗 4.1 Основа: 8-мерное векторное квантование с общими кодовыми книгами

В двух словах. Каждые 8 последовательных весов в строке маршрутизируемых экспертов становятся одним 12-битным (или 13-битным) индексом в кодовой книге, умноженным на коэффициент усиления для каждой строки.

Интуиция. Официальными экспертами являются FP4 с одной шкалой на 32 веса (4,25 бита / вес). Приведенный ниже скалярный формат, за который все равно приходится платить за эти масштабы — 0,25 бита / вес сам по себе. Векторное квантование кодирует 8 весов совместно, что превосходит скалярное искажение скорости при том же бюджете, и для этого требуется только одно усиление на строку. Здесь этого достаточно, потому что внутри ряда официальные шкалы различаются всего в 2 раза в среднем и в 4 раза в худшем случае (измеряется на слоях 0, 20 и 39).

Формат.

$$w[r, 8j : 8j+8] \approx g_r \cdot C[\text{idx}[r, j]] \quad C = 4096 \times 8 \text{ (12-bit)}$$
$$\text{bits per weight} = 12/8 = 1.5 \quad \text{or} \quad 13/8 = 1.625, \quad \text{plus one f16 gain}$$

Тренируем кодовую книгу.

1. Нормализовать каждую строку по среднему квадратичному отклонению, g_r .
2. Выполнить итерации алгоритма Ллойда на графическом процессоре. Этап присвоения значений объединен с другими этапами: кодовая книга передается через общую память блоками по 512 элементов, и каждый поток сохраняет свой текущий argmin в регистрах, поэтому матрица расстояний никогда не хранится в памяти (наивный $G = V \cdot C^T$ алгоритм с использованием cuBLAS перемещал 144 ГБ данных на каждого эксперта). Ширина вектора является параметром шаблона, поэтому 8 значений остаются в регистрах, что в 3 раза быстрее, чем при использовании ширины, определяемой во время выполнения.
3. Детерминированный по своей сути: равноудаленная начальная выборка (без источника случайных чисел), фиксированное количество раундов, атомарные операции с двойной точностью при обновлении. Одни и те же входные данные, одни и те же байты.

4. В финальном присваивании используется кодовая книга, а коэффициенты уже округлены до сохраненной точности, поэтому ошибка, о которой сообщает квантователь, — это именно та ошибка, которую увидит движок.

По одной кодовой книге на слой, общей для всех 384 экспертов и всех трех матриц. Поскольку эксперты статистически идентичны (относительная погрешность одинакова в пределах 1,9 %), кодовая книга для каждого слоя, обученная на наборе из ≥ 1000 векторов на кодовое слово, практически ничего не стоит: ошибка на 20-м слое — 0,17062 против 0,17072 при использовании одной кодовой книги на эксперта (немного *лучше*), на 39-м слое — +0,33 %, +0,15 % в среднем. Удаляется 15 744 кодовых книги — **3,09 ГБ**. Размер пула имеет значение: пул в 4 раза меньше потерял 0,96 % на 13-битных слоях.

Кодовые слова FP8 (E4M3). Хранение кодовых слов в формате E4M3 приводит к ошибке +0,12 % и вдвое сокращает количество байтов для поиска. Декодер должен использовать аппаратное преобразование (`cvt.in.f16x2.e4m3x2, sm_89+`); скалярный преобразователь обеспечивает точность до бита, но на 26 % медленнее при декодировании и на 20 % медленнее при предварительном заполнении.

Куда деваются сохраненные 3,09 ГБ: 14 самых мелких слоев перемещаются с 12 на 13 бит (ошибка -16,1%). Кривая ошибок выпуклая (удаление бита стоит 5,3 п.п., добавление одного дает 2,6–3,7 п.п.), поэтому дополнительные байты должны быть распределены тонким слоем, а не уложены в стопку: 14 слоев по 13 бит устраняют больше общей ошибки, чем 7 слоев по 14 бит, а неглубокие слои, по оценкам, стоят в 1,2–1,4 раза больше за ГБ, чем глубокие.

Схема расположения битовых плоскостей «12 + 1». Прямой 13-битный поток делает каждый блок с индексом 256 состоящим из 104 слов = 3,25 строк кэша, то есть каждая 128-байтовая загрузка охватывает две строки. На GB10 скорость загрузки, не заполняющей целые 128-байтовые строки, снижается с ~219 ГБ/с до ~142 ГБ/с (измерено с помощью нашего `mem_ceiling` зонда). Таким образом, 13-битные слои сохраняют 12-битный основной поток с 12-битной геометрией (один блок = 3 полные строки) плюс отдельный 1-битный слой: одна дополнительная 32-битная загрузка на блок и одно перемешивание потока на раунд.

Громкий провал. Полезные нагрузки версии 3 содержат новую магию (DQV3), а загрузчик вносит форматы в белый список. Движок, который не знает формат, отклоняет файл при загрузке; движок, который появился раньше белого списка, выводит экспертов с нулевыми значениями из первого предложения. Ни то, ни другое не может привести к правдоподобному мусору.

Нулевой корпус. База квантуется только на основе весов. Калибровка с учетом финансовых показателей не учитывает их и обходится в 2,4 стр. текста: знания о предметной области относятся к пункту ②, а не к пункту ①.

Из чего состоит 113,56 ГБ.

Часть	Формат	ГБ
Маршрутизированные-экспертные индексы	VQ-8, 26 слоев × 12 бит + 14 слоев × 13 бит	104.90
Выигрыш в строке	f16	0.30
Кодовые книги	40 × E4M3	0.003
Внимание, коллеги-эксперты, выходим на финишную прямую	q4_K	4.96
Спекулятивные осадочные башни	VQ-8, 12 бит	2.56
Все остальное	как отправлено	0.85

Улики. По сравнению с базовым уровнем того же размера (одна кодовая книга на эксперта, все уровни 12-разрядные): те же топ-1 **+1,9 п.п.** (финансы) и **+2,5 п.п.** (английский); наибольший прирост получили 5% худших позиций ($\Sigma \min p5 +22\% / +83\%$). В английском языке одна только база превосходит эту базовую, с ее коляской.

🔗 **4.2 Доменная сайдкап (反修)**

Мы называем решение **反修** («восстановление в обратном порядке») и его результат **усилителем** (放大器); сайдкап — это каталог с файлами усилителя.

В одном предложении. Для каждого эксперта повторно настройте один коэффициент усиления на каждый выходной канал его нисходящей проекции, а также один коэффициент смещения маршрутизатора для каждого эксперта, чтобы каждый квантованный блок МоЕ воспроизводил исходный выходной сигнал блока на основе текста домена.

Почему не аддитивная коррекция, а прирост. Коррекция, представляющая собой линейную функцию входных данных блока, например $y += B \cdot A \cdot x$ низкого ранга, имеет эффект первого порядка на выходе модели, равный ≈ 0 : она устраняет 9% ошибок на контрольных слоях, но при этом ухудшает сквозные метрики (что подтверждается сканированием с изменением размера шага). Коэффициент усиления для каждого эксперта добавляет $\Delta = \sum_e rw_e \cdot (s_e - 1) \odot y_e$, что зависит от того, каких экспертов выбрал маршрутизатор, а также от скрытого состояния каждого эксперта — не линейной функции от x — и влияет на результат.

Цель для каждого слоя.

$$y_i = \sum_k rw_{ik} \cdot (s_{\{e_{ik}\}} \odot y_{e_{ik}}) + y_{sh_i} \quad \# \text{ the engine}$$

y_e = expert down-projection output before the routing weight, y

$$\min_s \sum_i \| y_i - y_i^{\text{orig}} \|^2 + \lambda \cdot \sum_d d_d \cdot \sum_e (s_e[d] - 1$$

- y^{orig} — результат работы исходной модели для того же блока на том же токене, полученный с помощью обучающего набора данных: Код DeepSeek с полной точностью, 40 слоев × на каждый токен обучающего набора данных, записывается один раз для каждого обучающего набора данных (14 ГБ для 8192 токенов).
- Усиления являются диагональными, поэтому 5120 выходных каналов независимы друг от друга; каждый из них представляет собой задачу наименьших квадратов с 384 неизвестными.
- Задача решается с помощью метода **Гаусса — Зейделя** для каждого эксперта (3 прохода). Каждый эксперт затрагивает только ~128 строк, направленных к нему, и сразу обновляет невязку — системы размером 384×384 никогда не формируются.

- Горб масштабируется на d_d , **среднюю энергию всех экспертов на канале d** . Использование собственной энергии каждого эксперта вместо того, чтобы позволять экспертам, которые редко используются, доходить до крайностей: на нулевом слое обучение дало +32,7 %, а на контрольном отборе — -2685 %. При использовании общей шкалы на том же слое контрольный отбор дал +27,0 %.

Пристраиваемся по пути следования, слой за слоем.

- y_e и y_{sh} берутся из хуков в собственных ядрах предварительной загрузки движка; во второй реализации ничего не пересчитывается. Проверяется каждый слой: $\text{round_bf16}(\sum w \cdot y_e + y_{sh})$ совпадает с выходным битом движка для 99,996 % значений, в противном случае выполнение останавливается.
- Слои $0 \rightarrow 39$ решаются за один проход. Слой k дополняется уже смонтированными слоями $< k$, поэтому он видит именно то, что будет видеть при развертывании.
- Для каждого слоя: вычисляем смещение маршрутизатора Δb , монтируем его, повторно захватываем, затем вычисляем коэффициенты усиления.
- $\lambda \in \{0.01, 0.1, 1, 10, 100, 1000\}$ Выбирается только для строк, не участвующих в обучении. Не участвующие в обучении строки стратифицируются по исходному окну (каждое 4-е окно из 128 токенов, четверть токенов), поэтому каждый под-источник представлен с обеих сторон.
- **Слой монтируется только в том случае, если его коэффициент усиления на не участвующих в обучении строках превышает 0,5%.** Это фильтр значимости, а не ограничение по размеру: выбор лучшего из шести значений λ дает небольшое положительное число даже при чистом шуме, а в последовательной цепочке один шумовой слой искажает все последующие слои. В конечном счете, если усиливать каждый слой, это ухудшает все четыре показателя.

Решение с прямой решеткой FP4. Приращения хранятся в FP4 (E2M1, по одному масштабу на 32). Решение с плавающей запятой и последующее округление привели бы к потере 11% коррекции, поскольку в E2M1 всего 8 масштабов. Таким

образом, каждый шаг метода Гаусса — Зейделя состоит из трех этапов: решение одного эксперта → проецирование $s - 1$ на решетку FP4 и непосредственная выдача 17-байтовых блоков → обновление невязки с помощью значения *решетки*. Ошибка округления Expert e компенсируется экспертами, решаемыми после нее, — обратная связь по ошибкам в стиле GPTQ, применяемая к одномерным коэффициентам усиления. Слой 0 теряет 0,27 процентных пункта коэффициента усиления, а не 11%; размер sidacar уменьшился **291 МБ → 38,6 МБ** без регрессии метрики. Записанный файл *является* решетчатыми кодами, которые перед сохранением были проверены с помощью собственного декодера движка.

Смещение маршрутизатора Δb . Для каждого слоя и эксперта накапливаем разницу в оценках выбора между исходным набором и нашим: если в исходном наборе был выбран e , а в нашем нет, добавляем $thr - v_e$; если в нашем наборе был выбран e , а в исходном нет, вычитаем $v_e - thr$ (thr = наша оценка выбора K -го эксперта). Усредняем, активируем эксперта только после ≥ 8 событий, масштабируем на $\alpha \in \{0.5, 1, 1.5, 2.5\}$ в зависимости от доли событий в исходном наборе, и записываем слой только в том случае, если доля событий в исходном наборе строго улучшилась. Δb добавляется только к оценке выбора; весовые коэффициенты смешивания не изменяются. Исходная маршрутизация пересчитывается на основе полноточных входных данных устройства с помощью собственного маршрутизатора bf16 модели; результат пересчета должен совпадать с выбором движка в $\geq 99\%$ строк.

Стоимость. Тот же решатель, те же гейты, по одному запуску на домен в Spark:

Коляска	Подходящий корпус	Выигрыш / смещение маршрутизатора	Размер	Учительский набор + 40- слойное решение
финансы	8192 токена, пять источников финансирования в Китае	39 / 27 слоев	40,8 МБ	27 + 73 min

Коляска	Подходящий корпус	Выигрыш / смещение маршрутизатора	Размер	Учительский набор + 40- слойное решение
код	15 360 токенов, 30 языков программирования × 512	39 / 29 слоев	40,8 МБ	31 + 124 min
закон	15 360 токенов, 6 источников × 2560 (3 на китайском, 3 на английском)	40 / 24 слоя	41,8 МБ	31 + 104 min
медицина	15 360 токенов, 6 источников × 2560 (3 на китайском, 3 на английском)	40 / 29 слоев	41,8 МБ	32 + 115 min
Естественные науки	15 360 токенов, 6 источников × 2560 (3 на китайском, 3 на английском)	40 / 29 слоев	41,8 МБ	32 + 104 min

Доказательства (Те же топ-1, парные прогоны; полные таблицы в §5.1). Финансы: рост 71,7 → 73,9 %; смещение в сторону роутеров, рост остался неизменным, +0,24 процентных пункта и KLD −2,5 % (английский +0,78 процентных пункта, KLD −3,8 %). Для каждого домена: код 78,6 → 82,3 %, юриспруденция 72,9 → 76,4 %, медицина 69,1 → 72,8 %, наука 71,7 → 74,9 %.

🔗 4.3 Файл после тренировки

В одном предложении. Преобразуйте фразу «в этой позиции модель должна была написать a , а не b » в линейное уравнение для коэффициентов эксперта последнего слоя, решите все такие уравнения вместе, зафиксировав все остальное, и сохраните результат в виде третьей таблицы коэффициентов.

Почему не обычные градиенты для тонкой настройки. При использовании градиента потерь SFT в качестве целевого показателя прирост не меняется (на

контрольных данных +0,04 %). Градиент потерь — это плотное направление, задаваемое эмбедингом целевого токена, в то время как прирост для каждого эксперта может только перевесить выходные данные эксперта по каналам: эти два направления почти ортогональны. Решение о том, будет ли оно изменено, зависит от одного скаляра — разницы между двумя логитами, — и эта разница в точности линейна по отношению к выходному значению последнего слоя MoE.

Формулировка.

- **Точка принятия решения i :** позиция, в которой правильный жетон a должен побить самый сильный *другой* жетон b . (Не "токен, написанный в неправильном ответе": в 81% случаев этот токен уже избит, и $\arg\max$ является третьим токеном.)
- **Маржа $m_i = \ell_a - \ell_b$.** При inv_i фиксированном коэффициенте выходной RMSNorm изменение Δ выходной мощности последнего слоя приводит к увеличению запаса на $\alpha_i \cdot \text{inv}_i \cdot \sum_d y_d (W_a - W_b)_d \cdot \Delta_d$ (y = вес нормы, W = выходная мощность) и Δ является линейным в таблице усиления.
- **Решите $r_i \cdot s = \tau - m^o_i$** для всех точек принятия решения (r_i = строка коэффициентов из приведенного выше выражения, τ = целевой запас), плюс **строки ограничений** (каждая вторая позиция сохраняет свой запас top-1 / top-2, вес ρ), плюс гребень λ , с сопряженным градиентом без матрицы и предобуславливателем Якоби — 1,97 Млн неизвестных, секунды.
- **С учетом FP4:** квантовать решение для решетки, повторно прогнозировать с сохраненными значениями, помещать точки принятия решений, которые опустились ниже τ , в активный набор, решать снова.
- **Захват на пути развертывания.** Поля должны измеряться там, где фактически принимается решение: запрос через предварительное заполнение, сгенерированный текст через путь декодирования (`--score-split P`). Решение, зафиксированное только на пути предварительного заполнения, меняет решение при подсчете очков, но не при генерации.
- **Безопасность:** ③ — это собственный каталог, который при загрузке объединяется с ② и сверяется с ним по отпечатку; кандидаты, не прошедшие проверку, перемещаются в `rejected/`, но никогда не удаляются.

Доказательства и статус — наименее проработанная часть; см. §8. По запросам на обучение точки принятия решений меняются **55% → 88%** (64 / 73), в то время как **99,66%** из 3268 других позиций сохраняют тот же топ-1; прогнозируемая маржа коррелирует **0,9995** с реальной. При свободной генерации обученный токен решения переворачивается, но модель может вернуться к исходному выводу примерно через 6000 символов: один токен перемещает одно предложение, а не всю цепочку рассуждений.

4.4 Двигатель

В одном предложении. Каждое ядро на «горячем» пути написано для этого формата на GB10, и при каждом ускорении выходные данные при температуре 0 должны быть идентичными.

Стена. Декодирование ограничено объемом памяти: каждый токен считывается ~ 6,3 ГБ. На GB10 измеряется ~ 235 ГБ / с, что составляет 26,8 мс (37 токенов / с). Мы работаем со скоростью 32,5 мс — **82% от стены**.

За каждый декодированный токен	Байты	Охват ядра
Выходная головка (q4_K)	372 МБ	~249 ГБ/с
Внимание и прогнозы на основе коллективного экспертного мнения (q4_K)	~ 3,6 ГБ	193–210 ГБ/ с
Маршрутизированные эксперты (VQ, 6 из 384 на слой)	1,64 ГБ	~155 ГБ/с
Проекция n-грамм памяти (FP8)	314 МБ	~226 ГБ/с
Фрезерный станок и небольшие матрицы для смешивания	314 МБ	132–136 ГБ/ с

Расшифровывать.

- **Экспертное ядро VQ.** Стойкий. Кодовая книга каждого слоя загружается в общую память один раз на SM. Потоки индексов считываются в виде блоков по 384 байта (3 полные строки по 128 байт), а коды распределяются с помощью перемешивания ворпов; последовательные строки передаются друг за другом. Во время загрузки каждая экспертная полезная нагрузка

сдвигается таким образом, чтобы ее индексный поток начинался с 128-байтовой границы (только копирование на устройство, файл не трогается): 145–155 → 124–129 мкс на слой.

- **q4_K GEMV** (остальные ~60 % байт). Два варианта: *stage* (СТА перемещает целые строки в общую память перед вычислениями) и *pipe* (постоянная двойная буферизация `sr.async` для групп размером 4–12 КБ). 22,9 → 18,8 мс на токен.
- **Цельношаговый граф CUDA.** Один захват на позиционный сегмент; позиция токена хранится в слоте устройства, поэтому при повторном воспроизведении захват не происходит. Временные буферы создаются *до* захвата: выделение памяти внутри захвата делает их недействительными, а без графов токен обходится в 43,9 мс.
- **Программный зависимый запуск.** 1391 ребро ядра: каждое ядро предварительно загружает свои постоянные веса, а затем ожидает своего производителя.
- **Дополнительный поток.** Ветка KV и общий эксперт работают параллельно с основной цепочкой.
- **Расширенный контекст.** Были переписаны ядра выбора разреженного внимания (1024-поточный top-k, пакетное чтение, 4 группы на warp): при 51 тыс. контекстов дополнительная стоимость снизилась с 4,6 до 2,8 мс на токен.

Предварительная обработка. Эксперты работают на тензорных ядрах bf16 (mma.m16n8k16). Кодовые слова E4M3 точно преобразуются в bf16, а активации уже находятся в сетке bf16, поэтому каждое произведение равно скалярному пути. При длительном накоплении в тензорном ядре теряются младшие биты (3,5 % выходных данных отличаются на один ulp), поэтому каждый срез k16 накапливается с нуля и добавляется с помощью отдельной операции FADD (0,5 %). Подсказка обрабатывается блоками по 2048 токенов: каждый блок должен один раз декодировать весь битовый поток эксперта слоя (2,55 ГБ на слой), поэтому размер блока определяет, сколько токенов амортизируют это декодирование. Реальная маршрутизация неравномерна (блок из 2048 токенов затрагивает 200–350 из 384 экспертов, при этом на самого популярного эксперта приходится 8–16 % всех выборов), поэтому один рабочий элемент охватывает до

128 токенов эксперта с 16 варпами, а 12-битные слои хранят свой кодовый словарь в общей памяти, уже преобразованный в bf16. Sparse attention — это однопроходное ядро: фрагменты запроса остаются в регистрах, оценки берутся из `mma.sync`, работающая функция `softmax` пересчитывает выходной аккумулятор на месте (расположение аккумулятора в `mma.m16n8k16` задокументировано, а расположение фрагментов `mma` — нет), и каждый ключ собирается только один раз. Строки памяти n-грамм для блока извлекаются в потоке ввода-вывода, пока графический процессор выполняет вычисления, и только последняя позиция последнего блока проходит через выходную головку. Подсказка на 12,5 тыс. токенов: **216,5 → 1055 токенов/с**. Качество не изменилось в пределах погрешности округления: набор финансовых суждений показывает $\Sigma_{\min} 0,7436 \rightarrow 0,7441$, KL $0,5149 \rightarrow 0,5131$, тот же top-1 $74,56\% \rightarrow 74,77\%$, и все, кроме ядра внимания, идентично скалярному пути.

Умозрительная расшифровка.

- Три черновые версии модели имеют одинаковый объем памяти VQ (2,56 ГБ).
- Пакет проверки ($1 + k$ строк) имеет собственные ядра: GEMV на этапе/канале с несколькими строками и постоянное экспертное ядро размером (уникальный эксперт $\times$ строки), которое считывает каждый выбранный бит эксперта один раз.
- **Планировщик.** Руководитель проекта сообщает о достоверности c_j для каждой позиции. При выживании $a_j = \prod_{i \leq j} \sigma(c_i)$ выберите

$$k^* = \operatorname{argmax}_k (1 + \sum_{j \leq k} a_j) / (c_{\text{draft}} + c_{\text{v1}} + c_{\text{tok}} \cdot (1 + k))$$

с тремя затратами, измеряемыми самим движком, на каждый запрос: время, затраченное на простой шаг графа, на черновой раунд и на проверку $1 + k$ строк (взвешенная линия наименьших квадратов на основе количества строк, увиденных на данный момент; подготовительные раунды, за которые отвечает ленивое выделение памяти, не учитываются). В первых раундах проверяется самый длинный блок, а затем одна строка, чтобы получить две точки, необходимые для построения линии. Констант затрат на этапе компиляции нет. Вероятности выживания также откалиброваны онлайн: $p =$

попадания в первую позицию / $\sum \sigma(c_0)$ по запросу, умноженные на каждый $\sigma(c_i)$. Меняется только время k , никогда токен: спекулятивный вывод идентичен обычному декодированию в байтах для каждого k , поэтому настенные часы с управлением k безопасны.

- **Гарантия:** при температуре 0 спекулятивный вывод идентичен побайтово простому жадному выводу, что проверяется при каждом изменении. По умолчанию включено как для жадных, так и для сэмплированных запросов. При сэмплированном запросе черновики берутся из собственного распределения черновых башен (с той же температурой и усечением), а их достоверность проверяется методом отбраковки на графическом процессоре (принять черновой токен с вероятностью $\min(1, p_{\text{target}}/p_{\text{draft}})$, в противном случае взять из остатка $\max(0, p_{\text{target}} - p_{\text{draft}})$), поэтому распределение токенов в точности совпадает с распределением при обычном сэмплированном декодировании. Решение о выборе из распределения, а не из argmax башен, было принято аналитическим жюри на основе фиксированных текстов (ожидаемый показатель принятия на первой позиции при температуре 1,0: $0,69 \rightarrow 0,74$ и $0,63 \rightarrow 0,76$), а не на основе времени выполнения, поскольку каждый выбранный текст — это отдельный случай. Калибровка показателя принятия планировщика выполняется онлайн для каждого запроса, без использования констант. Сама выборка выполняется на графическом процессоре внутри графа декодирования (Gumbel-max с точным $\text{top}_k/\text{top}_p/\text{min}_p$ усечением по основанию; хост считывает 16 байт на строку), с проверкой распределительным вентилем (`make cuda-regression`).
- **Почему не выше:** каждый дополнительный проверенный токен затрагивает ~3,9 новых экспертов (хеш-подобная маршрутизация, §2.2), поэтому проверка $k + 1$ токена обходится гораздо дороже, чем проверка одного. Пакетная проверка выполняется с использованием 57 % собственной «байтовой стены»; это следующий шаг.

Память. Значения поддерживаются в формате mmap. Состояние для каждого запроса растет вместе с фактически используемыми позициями: 1 МЛН KB сам по себе составляет 0,89 ГБ, а скретч для каждого последующего запроса увеличивается вдвое вместо того, чтобы быть размером на 1 МЛН заранее (2,5 ГБ

→ 160 МБ для запроса на 40 тыс. токенов). Логиты и ошибка в заголовке вывода больше не масштабируются с блоком предварительного заполнения (запрос на генерацию сохраняет 10 строк логитов, а не 2048). Запрос 106,7k-токена через сервер оставляет свободным 6,7 ГБ на компьютере объемом 121 ГБ. Нет регулятора контекста — привязка берется из метаданных GGUF и --ctx отклоняется.

🔗 4.5 Как мы проводим измерения

В одном предложении. Единственная метрика — это исходная модель, запущенная с использованием собственного кода DeepSeek на полную мощность, и наш движок оценивается по тем же критериям.

- **Учительница.** Официальный код вывода PyTorch от DeepSeek с оригинальными весами FP4 / FP8, передаваемыми послойно с SSD-накопителя (510 ГБ не помещаются в память). Логиты учителей кэшируются для каждого (текст, длина).
- **Студенты.** Путь оценки движка (--score-ids), тот же формат файла, один компаратор (anchor_metrics) для всего.
- **Пять показателей.** Один и тот же top-1; Σ min (среднее, медиана, p5); среднее значение KL(исходный текст || наш текст); коэффициент PPL. Σ min и KL являются основными показателями. Один и тот же top-1 — это слишком мягко: простой текст скрывает повреждения (один и тот же файл может иметь показатель 0,90 для простого кода и 0,52 для сложного текста).
- **Несовпадающие срезы.** Текст каждого домена разделяется на подходящий и оценивающий срезы, которые никогда не пересекаются; источник (документ, репозиторий кода) попадает только в один из них.

Линейка	Оценивающий срез	Оригинальная модель PPL
финансы	8192 токена, пять источников финансирования в Китае	6.978

Линейка	Оценивающий срез	Оригинальная модель PPL
код	15 360 токенов: 30 языков × 512 из GitHub (codeparrot/github-code-clean), заголовки лицензий удалены	4.766
закон	15 360 токенов, 6 × 2560: законы Китая, факты уголовных дел (CAIL2018), вопросы экзамена на адвоката (JEC-QA); решения Верховного суда США, законодательство ЕС, договорные условия (LexGLUE)	4.865
медицина	15 360 токенов, 6 × 2560: вопросы и ответы из китайской медицинской энциклопедии, консультации реальных врачей (cMedQA2), вопросы для экзамена на получение лицензии с пояснениями (CMExam); аннотации PubMed, отчеты о клинических случаях в PMC, вопросы USMLE (MedQA)	8.625
Естественные науки	15 360 токенов, 6 × 2560: аннотации к китайским журналам, STEM и гуманитарные / социальные науки (CSL), вопросы по STEM для колледжей (C-Eval); исходный код arXiv в LaTeX, полные статьи S2ORC (peS2o), вопросы по STEM от MMLU-Pro	8.770
Английский	WikiText-2, 512 токенов	1.657

- **Парные, по одной переменной за раз.** При сравнении используются одни и те же бинарные значения в одном и том же раунде.
- **Гейты, выходящие за рамки пяти чисел.**
 - Идентичность байта с температурой 0: прямой запуск == граф CUDA == воспроизведение, спекулятивный == обычный.
 - Изменения формата также проверяются на этапе декодирования. Пять чисел вычисляются на этапе предварительного заполнения и не могут обнаружить ошибку декодирования ядра: 13-битное ядро, продвигающее указатель битовой плоскости по блокам, а не по

индексам групп, оставляет их зелеными, в то время как PPL на этапе декодирования считывает 8,19 вместо 5,80.

- Завершение работы определяется без ограничения на вывод; при ограничении на вывод может отображаться только надпись «еще не остановлено».

[5. Результаты](#)

[5.1 Качество](#)

Каждый домен оценивается по своему судейскому срезу ([\\$4.5](#)): без sidocar и с sidocar этого домена. В таблице все строки получены из одного и того же бинарного файла движка за один запуск, кроме строк, отмеченных ¹.

Финансы (судейский срез, 8192 токена; исходная модель PPL 6.978):

	Тот же топ-1	$\Sigma \min$ (медиана / p5)	Средний KL	Коэффициент PPL
① только основание ¹	71.73%	0.703 (0.751 / 0.231)	0.612	1.339
① + финансовая поддержка, только прибыль	73.94%	0.742 (0.810 / 0.276)	0.522	1.282
① + финансовая поддержка	74.57%	0.745 (0.810 / 0.283)	0.512	1.267

¹ Ссылка на предыдущий файл; разница между движком и ссылкой на тот же файл составляет KL 0,013. Предварительное заполнение тензорного ядра ([\\$4.4](#)) сдвигает строку finance-sidocar на 74,77 % / 0,744 / 0,513 (шум округления).

Код (судейский срез, 15 360 токенов = 30 языков × 512; исходная модель PPL 4.766):

	Тот же топ-1	$\Sigma \min$ (медиана / p5)	Средний KL	Коэффициент PPL
① только основание	78.61%	0.754 (0.809 / 0.318)	0.421	1.367
① + кодировка sidecar	82.26%	0.803 (0.872 / 0.402)	0.303	1.229

Закон (судейский срез, 15 360 токенов = 6 источников × 2560; исходная модель PPL 4.865):

	Тот же топ-1	$\Sigma \min$ (медиана / p5)	Средний KL	Коэффициент PPL
① только основание	72.90%	0.713 (0.773 / 0.206)	0.578	1.358
① + законный прицеп	76.36%	0.760 (0.834 / 0.272)	0.454	1.251

Медицина (судейский срез, 15 360 токенов = 6 источников × 2560; исходная модель PPL 8.625):

	Тот же топ-1	$\Sigma \min$ (медиана / p5)	Средний KL	Коэффициент PPL
① только основание	69.08%	0.683 (0.692 / 0.259)	0.602	1.378
① + аптечка	72.82%	0.739 (0.767 / 0.325)	0.465	1.280

Science (судейский срез, 15 360 токенов = 6 источников × 2560; исходная модель PPL 8.770):

	Тот же топ-1	$\Sigma \min$ (медиана / p5)	Средний KL	Коэффициент PPL
① только основание	71.65%	0.707 (0.726 / 0.282)	0.537	1.259
① + научный сайдкар	74.93%	0.753 (0.788 / 0.347)	0.422	1.173

General English (WikiText-2, 512 токенов; исходная модель PPL 1.657): проверка того, что сайдкар домена не снижает общую производительность. Строка bare-base выглядит одинаково во всех прогонах.

	Тот же топ-1	$\Sigma \min$ (медиана / p5)	Средний KL	Коэффициент PPL
① только основание	78.52%	0.769 (0.950 / 0.065)	0.694	1.797
① + финансовая поддержка	80.66%	0.787 (0.965 / 0.083)	0.619	1.658
① + кодировка sidecar	82.23%	0.797 (0.972 / 0.103)	0.584	1.644
① + законный прицеп	82.81%	0.800 (0.973 / 0.085)	0.580	1.590
① + аптечка	82.81%	0.801 (0.976 / 0.102)	0.599	1.592
① + научный сайдкар	82.62%	0.800 (0.975 / 0.126)	0.566	1.564

При 512 позициях одна позиция составляет 0,2 п.п.: эти строки читаются как «без регрессии», а не как выигрыш.

🔗 5.2 Скорость (один DGX Spark)

Рабочая нагрузка	Предварительная заливка	Расшифровывать
Подсказка на 12,5 тысяч токенов	1,055 т/с (539 с -- decoder-full)	—
Запрос реального агента, подсказка с 14,1 тыс. токенов	940 т/с	—
Запрос на 106,7 тыс. токенов через сервер	671 т/с (159 с до первого токена)	—
Короткая подсказка, обычная жадность	—	30,5–30,7 т/с
Запрос реального агента, подсказка с использованием 14,1 тыс. токенов, обычный текст	—	28,9–29,4 т/с
Тот же запрос, спекулятивный (по умолчанию)	—	43,0 т/с (3,04 токена за раунд)
Запрос никогда не использовался для настройки, подсказка 9,2 тыс. слов, умозрительная	—	40,0 т/с
51 тыс. контекстов, обычный	—	27,5 т/с

6. Что не сработало

Отрицательные результаты так же важны для дизайна, как и положительные. Каждый ряд был измерен, а не просто упомянут.

Идея	Что мы измерили	Вердикт
Динамические биты на слой / эксперт / матрицу	Вариативность 3% / 1,25× / 33:33:33; при небольшом сдвиге требуется ≥ 1,189×	Оптимальная форма одежды
Динамические биты в строке	2,5-кратный энергетический разброс, обусловленный стоимостью кодовой	Отвергнутый

Идея	Что мы измерили	Вердикт
	книги и разрядностью целых чисел: от −1,6% до −5,9%	
Профессиональная обрезка для достижения объема 100 ГБ	Оставил бы по 130 из 384 экспертов в каждом слое	Отказано: скрытое решение по домену
Аддитивная коррекция низкого ранга $y \leftarrow y + B \cdot A \cdot x$	Выдержка +9% на слой, сквозная отрицательная; эффект первого порядка на выходе ≈ 0	Вся семья отвергнута
Средняя / смещенная коррекция ошибки квантования	Внутри слоя +4 pp; на выходе −0,33 pp (3 слоя); 40 слоев 70,52 $\rightarrow$ 63,89 %	Отвергнутый
Повторное решение кодовых книг на основе данных предметной области	65,43 % против 65,59 %	Никакой выгоды
Энтропийное кодирование индексов VQ	Эмпирическая энтропия 11,927 из 12 бит: экономит 0,6%	Оно того не стоит
VQ с ограничением по энтропии + потоки переменной длины	$\approx +0,1$ бит после служебных данных потока	Оно того не стоит
Финансовая база	Финансовая стабильность, общий текст −2,4 стр.	База остается без корпуса
Сокращение словарного запаса до финансовых терминов	От +0,2% до −54% на невидимом запросе	Отвергнутый
Сгруппированное ядро эксперта по мультитокенам для верификации	Сокращает вдвое время поиска в кодовой книге, экономия 0 мс	Отвергнутый
Вторая машина для декодирования	Разделение слоев −19% (вверх по течению); тензорное распараллеливание 1,25×	Одна коробка

🔗 7. Запустите его

То, что вам нужно.

- NVIDIA DGX Spark (GB10, sm_121, 128 ГБ) — единственная протестированная машина. Для кодовой книги FP8 требуется sm_89 или новее. Версия 4.1 работает только на CUDA.
- Linux aarch64 с исполняемым файлом CUDA 13 (`libcudart.so.13`, `libcublas.so.13`, `libcublasLt.so.13`; они поставляются с DGX OS). Отсутствующие библиотеки отображаются как `error while loading shared libraries: libcudart.so.13`.
- ~320 ГБ на локальном SSD: 113,6 ГБ для этого репозитория + 203 ГБ для двух официальных шардов (см. ниже).
- Больше ничего не нагружает систему: движок отказывается запускаться, если не укладывается в бюджет в 110 ГБ.

Одна команда. В Spark:

```
curl -fsSL0 https://huggingface.co/wenzhouwu/YoungAi-DeepSeek-V4.1-Flash
bash install.sh
```

Он проверяет оборудование (графический процессор, библиотеки CUDA 13, память, диск), загружает этот репозиторий и два официальных шарда (~317 ГБ), собирает базовую модель, запускает сервер на `127.0.0.1:8000` с помощью финансового сайдкара под контролем памяти и выводит ответ на тестовый вопрос в одну строку. Все выполняется под `--dir`; без `sudo`. Прервано? Запустите ту же команду еще раз: готовые файлы будут пропущены, и сборка возобновится с того места, где она остановилась.

```
      # install and serve with the coding sidecar
:t --domain finance    # switch domains (all sidecars are already local)
```

Вариант	Эффект
<code>--domain finance code law medicine science none</code>	какую ② дополнительную библиотеку загружает сервер (по умолчанию <code>finance</code> ; <code>none</code> = bare base)
<code>--dir DIR</code>	каталог установки (по умолчанию <code>~/youngai</code>)
<code>--engram-dir DIR</code>	у вас уже есть <code>model-00047-of-00048.safetensors</code> и <code>model-00048-of-00048.safetensors</code> из официального чекпоинта в DIR: используйте их вместо них, не загружайте 203 ГБ
<code>--host 0.0.0.0 / --port N</code>	обслуживает вашу локальную сеть / другой порт
<code>--endpoint https://hf-mirror.com</code>	загрузка через зеркало
<code>--no-xet</code>	загрузка через обычный канал LFS (если передача данных продолжает прерываться из-за «закрытия соединения одноранговым узлом»)
<code>--posttrain</code>	Также загрузите экспериментальный файл после обучения (только для финансовой модели)
<code>--no-start</code>	только установить; позже <code>bash install.sh start stop status</code>

Ниже приведены шаги, которые выполняет скрипт, чтобы сделать это вручную.

Файлы в этом репозитории.

Путь	Что
<code>DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative.gguf.part01-of-40 ... part40-of-40</code>	① база, 113 556 639 424 байт, разделена на 40 частей
<code>SHA256SUMS</code>	sha256 собранной базы и каждой ее части
<code>install.sh</code>	установщик с одной командой, описанный выше

Путь	Что
DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative-grrb-vqfin41_vqhalf_a_n8192-engine/	② финансовая вспомогательная модель: <code>gr_Lnn.bin</code> (коэффициенты усиления, 39 слоев) + <code>rb_Lnn.bin</code> (смещение маршрутизатора, 27 слоев) + <code>manifest.txt</code> (λ для каждого слоя и коэффициент усиления)
DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative-grrb-code_fit_n15360-engine/	② кодирующая вспомогательная схема, та же компоновка: усиления на 39 слоях, смещение маршрутизатора на 29 слоях
DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative-grrb-law_fit_n15360-engine/	② закон о параллельных вычислениях: прирост на 40 слоях, смещение на 24 слоях
DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative-grrb-med_fit_n15360-engine/	② медицинская коляска: усиление на 40 слоях, смещение на 29 слоях
DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative-grrb-sci_fit_n15360-engine/	② Научный сайдкар: прирост на 40 слоях, смещение маршрутизатора на 29 слоях
posttrain-experimental-20260924/	③ экспериментальный файл после обучения: <code>gr_L39.bin</code> + <code>base.fnv</code> (см. ниже)
bin/ds4, bin/ds4-server	двоичные файлы движка, созданные на основе Spark с помощью <code>make cuda-spark</code>
LICENSE, LICENSE-DeepSeek	Уведомления MIT для движка (вкл. GGML) и весов модели

Шаг 1 — скачать и собрать. При сборке вручную потребуется 227 ГБ свободного места (установщику нужна одна часть, потому что он добавляет и удаляет по одной части за раз).

```
hf download wenzhouwu/YoungAi-DeepSeek-V4.1-Flash --local-dir ds4-v41
cd ds4-v41
sha256sum -c --ignore-missing SHA256SUMS # every part must :
cat DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative.gguf.part{01..40}-of-40 :
sha256sum -c --ignore-missing SHA256SUMS # now the assembled
```

```
rm DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative.gguf.part*-of-40
chmod +x bin/ds4 bin/ds4-server # downloads do not
```

Шаг 2 — таблицы памяти n-грамм. Их нет в этом репозитории: движок считывает их в исходном виде из двух фрагментов официального чекпоинта ($\approx 101,5$ ГБ каждый).

```
.-Flash \
model-00048-of-00048.safetensors --local-dir /data/DeepSeek-V4.1-Flash
```

Подойдет любая папка. Укажите движку, где она находится, с помощью `--engram-dir` (следующий шаг). Сам GGUF записывает только путь, по которому находились эти осколки на компьютере, на котором он был создан. На вашем компьютере такого пути нет — без `--engram-dir` движок выдает предупреждение при загрузке и останавливается при первом запросе с ds4: `engram 表打不开 ...` («не удается открыть таблицу инграмм»).

Шаг 3 — подавайте.

```
h-vq8sh14-q4k-mtpnative-grrb-vqfin41_vqhalf_a_n8192-engine # finance
h-vq8sh14-q4k-mtpnative-grrb-code_fit_n15360-engine # code
h-vq8sh14-q4k-mtpnative-grrb-law_fit_n15360-engine # law
h-vq8sh14-q4k-mtpnative-grrb-med_fit_n15360-engine # medicine
h-vq8sh14-q4k-mtpnative-grrb-sci_fit_n15360-engine # science
DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative.gguf --zchain $SIDECAR \
DeepSeek-V4.1-Flash --mem-budget-mb 110000 --host 0.0.0.0 --port 8000
```

Загрузка занимает около двух минут. Конечные точки: `/v1/chat/completions`, `/v1/completions`, `/v1/responses` (в стиле OpenAI) и `/v1/messages` (в стиле Anthropic). Регуляторы выборки, которые не используются в запросе, соответствуют рецепту модели: `temperature 1.0`, `top_p 1.0`, без `min_p` — выборка на графическом процессоре, спекулятивное декодирование включено (34–37 токенов/с при выполнении запроса агента на 14 000 токенов по сравнению с 45,5 при жадном декодировании). Для получения воспроизводимого по байтам вывода отправьте `"temperature": 0` явно. По умолчанию запросы выполняются

в **неинтеллектуальном** режиме. Включить интеллектуальный режим для каждого запроса с помощью "reasoning_effort": "high" (или "max"), "thinking": {"type": "enabled"}, "think": true, или "model": "deepseek-reasoner". Квантованные весовые коэффициенты недооценивают решения о «столе» (</think>, EOS) по сравнению с моделью FP, поэтому сегмент с длительным интеллектуальным анализом может не завершиться. Неинтеллектуальный режим является безопасным по умолчанию до тех пор, пока интеллектуальный режим не будет восстановлен. Не делайте жадность настройкой по умолчанию в чате: на вопросы, на которые есть очень мало правильных ответов, чат может бесконечно заикливаться в разделе «Размышления» (например, «Назовите 5 китайских идиом, оканчивающихся на 五»).

Командная строка.

```
./bin/ds4 --cuda -m DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative.gguf --zchain
--engram-dir /data/DeepSeek-V4.1-Flash -p "Explain the price-to-earnings ratio"
# samples with the model card's recipe by default (plain decode); add --no-dspark
# plus --no-dspark for plain greedy (speed baselines); drop --zchain to
```

Файл после обучения — это эксперимент, а не обновление. posttrain-experimental-20260924/ содержит выигрыши последнего слоя, полученные на точках принятия решений по одному запросу на прогноз рынка; он демонстрирует формат и путь загрузки. base.fnv — это отпечаток финансового сайдкара, поэтому он работает только с этим сайдкармом, и движок отклоняет любые другие сочетания:

```
./bin/ds4-server ... --zchain DeepSeek-V4.1-Flash-vq8sh14-q4k-mtpnative-  
--posttrain posttrain-experimental-20260924
```

Исходный код движка, квантификатора и решателей пока не опубликован.

- **Пост-обучение (③)** — это рабочий механизм, но еще не готовый продукт. Он меняет целевые решения по запросам, по которым было принято решение, не затрагивая другие токены, но не переносит их: в дни, когда модель не обучалась, точки принятия решений остаются на уровне 55% → 55%. Один измененный токен меняет одно предложение, а не всю цепочку рассуждений.
- **Пять областей.** Финансы, код, право, медицина и наука (§5.1), все пять сайдкаров в этом репозитории. Все модели оцениваются по принудительному тесту, ни одна из них не тестировалась на сквозных задачах (агентное кодирование, ответы на юридические или клинические вопросы).
- **Только CUDA, протестировано на одном типе компьютера.** V4.1 не работает на Metal.
- **Спекулятивное декодирование работает только в режиме жадного декодирования.** Запросы на выборку возвращаются к обычному декодированию.
- **Prefill на тензорных ядрах не является битовой идентичной** версией объединенного скалярного пути (экспертов) или двухпроходного ядра внимания (однопроходного мгновенного внимания); оба различия находятся на уровне шума округления (§4.4).

🔗 Автор и контактное лицо

Вэньчжоу Ву (310066827@qq.com)

Вопросы, отчеты о воспроизведении и предложения о сотрудничестве приветствуются.

🔗 Благодарность

Этот проект начинался как форк [antirez/ds4](https://github.com/antirez/ds4) (DwarfStar) — специфичного для DeepSeek-V4 движка от Сальваторе Санфилиппо и других участников, в котором

хранится документация по путям V4 / Metal. Как и в основной ветке, мы многим обязаны [llama.cpp](#) и [GGML](#): от них мы взяли GGUF, схемы квантования, такие как q4_K, и многое из того, что с таким трудом было разработано в ядре, а уведомление об авторских правах авторов GGML осталось в LICENSE. Модель принадлежит DeepSeek. Спасибо DeepSeek за публикацию весов и эталонного кода логического вывода, который служит для нас эталоном.

Лицензия

Движок: MIT — см. [LICENSE](#). Квантованные веса взяты из DeepSeek V4.1 Flash, MIT — см. [LICENSE-DeepSeek](#).

 Системная тема

Компания

Правила использования

Конфиденциальность

О нас

Вакансии

Веб-сайт

Модели

Наборы данных

Пространства

Цены

Документы

